

AutoMapper vs Mapperly

Side-by-side mapping examples · .NET 8 / C# · for Luís

Both approach the same problem — turning a **User** entity into a **UserDto** — but Mapperly resolves everything at compile time via a source generator, while AutoMapper resolves it at runtime via reflection/expression trees. The C# on the Mapperly side is close to what you'd write by hand; it's just generated for you.

1. The model classes (shared by both examples)

```
public class User
{
    public int Id { get; set; }
    public string FirstName { get; set; }
    public string LastName { get; set; }
    public DateTime BirthDate { get; set; }
    public UserStatus Status { get; set; }
    public Address Address { get; set; }
    public string PasswordHash { get; set; }
}

public class Address
{
    public string City { get; set; }
    public string Country { get; set; }
}

public enum UserStatus { Active, Inactive, Suspended }

public class UserDto
{
    public int Id { get; set; }
    public string FullName { get; set; }    // composed from First+Last
    public int Age { get; set; }           // computed from BirthDate
    public string Status { get; set; }     // enum -> string
    public string City { get; set; }       // flattened from Address.City
}
```

2. Basic mapping + DI registration

AUTOMAPPER	MAPPERLY
<pre>// Profile.cs public class UserProfile : Profile { public UserProfile() { CreateMap<User, UserDto>() .ForMember(d => d.FullName, opt => opt.MapFrom(s => \$"{s.FirstName} {s.LastName}")) .ForMember(d => d.Age, opt => opt.MapFrom(s => DateTime.Now.Year - s.BirthDate.Year)) .ForMember(d => d.Status, opt => opt.MapFrom(s => s.Status.ToString())) .ForMember(d => d.City, opt => opt.MapFrom(s => s.Address.City)); } } // Program.cs builder.Services.AddAutoMapper(typeof(UserProfile)); // Usage (constructor-injected) public class UserService(IMapper mapper) { public UserDto ToDto(User u) => mapper.Map<UserDto>(u); }</pre>	<pre>// UserMapper.cs [Mapper] public partial class UserMapper { [MapProperty(nameof(User.Address), nameof(Address.City)], nameof(UserDto.City)))] // FullName combines 2 source props, so we // point Mapperly at a method explicitly via // MapPropertyFromSource + Use. It is NEVER // auto-detected just by matching types. [MapPropertyFromSource(nameof(UserDto.FullName), Use = nameof(BuildFullName))] // Age is derived from ONE source prop but // needs a type transform (DateTime -> int), // so it also needs an explicit Use pointer. [MapProperty(nameof(User.BirthDate), nameof(UserDto.Age), Use = nameof(BuildAge))] public partial UserDto ToDto(User user); private string BuildFullName(User u) => \$"{u.FirstName} {u.LastName}"; private int BuildAge(DateTime birthDate) => DateTime.Now.Year - birthDate.Year; } // Program.cs builder.Services.AddSingleton<UserMapper>(); // Usage (constructor-injected) public class UserService(UserMapper mapper) { public UserDto ToDto(User u) => mapper.ToDto(u); }</pre>

Mapperly maps **Id** automatically (same name, same type). It **never** guesses a custom method just because the types line up (e.g. a method that takes a `User` and returns `string`) — that would be ambiguous if you had several such methods. For anything combining/deriving values like **FullName**, you must explicitly point Mapperly at the method using **[MapPropertyFromSource(..., Use = nameof(Method))]**. Flattening (**Address.City** → **City**) needs an explicit **[MapProperty]** attribute in Mapperly (AutoMapper does it by convention automatically).

3. Ignoring a property (e.g. never expose PasswordHash)

AUTOMAPPER	MAPPERLY
<pre>CreateMap<User, UserDto>() .ForMember(d => d.PasswordHash, opt => opt.Ignore()); // Note: UserDto has no PasswordHash // property in our example, so this line // wouldn't even be needed here — but if // the DTO DID have a field you never // want auto-populated, this is the call.</pre>	<pre>[Mapper] public partial class UserMapper { [MapperIgnoreSource(nameof(User.PasswordHash))] public partial UserDto ToDto(User user); } // MapperIgnoreSource = "I know this source // property exists, don't warn me it's unused" // MapperIgnoreTarget = "don't populate this // destination property at all"</pre>

4. Enum mapping

AUTOMAPPER	MAPPERLY
<pre>// Enum -> string: works out of the box // via ToString(), or explicit ForMember // as shown in section 2. // Enum -> different Enum needs manual // per-value mapping if names differ: CreateMap<UserStatus, ContractStatusDto>() .ForMember(d => d, opt => opt.MapFrom(s => s == UserStatus.Active ? ContractStatusDto.Ok : ContractStatusDto.NotOk));</pre>	<pre>// Enum -> string: automatic, no code needed // Enum -> different Enum, mapped by NAME // automatically. For custom value mapping: [Mapper] public partial class UserMapper { [MapEnumValue(UserStatus.Active, ContractStatusDto.Ok)] [MapEnumValue(UserStatus.Suspended, ContractStatusDto.NotOk)] public partial ContractStatusDto ToContract(UserStatus status); } // Bonus: Mapperly WARNs at compile time if // an enum member has no mapping target.</pre>

5. Catching mistakes: runtime vs compile time

Scenario	AutoMapper	Mapperly
You add a new property to UserDto but forget to map it	Silently maps as null/default, or throws AutoMapperMappingException only when .AssertConfigurationIsValid() runs (often only in a test)	Compiler warning immediately in the IDE, before you even build (RMG012: unmapped member)
Debugging a wrong mapping	Step into reflection/expression-tree internals — hard to follow	Open the generated UserMapper.g.cs file and read plain, steppable C#
App startup cost	Builds/validates the mapping configuration at startup	None — mapping code already exists as compiled IL

Practical tip: right-click your Mapperly-generated file in the IDE (or look under obj/Debug/net8.0/generated/...) the first time you use it. Seeing the real generated C# is the fastest way to build trust that it's doing what you expect — it removes the "black box" feeling AutoMapper can have.

6. Quick syntax cheat-sheet

Need	AutoMapper	Mapperly
Declare a mapping	CreateMap<S,D>()	partial D Method(S s);
Rename/redirect property	.ForMember(...).MapFrom(...)	[MapProperty(nameof(S.X), nameof(D.Y))]
Ignore a target property	.ForMember(d=>d.X, opt=>opt.Ignore())	[MapperIgnoreTarget(nameof(D.X))]
Ignore a source property	(not usually needed)	[MapperIgnoreSource(nameof(S.X))]
Custom conversion logic	Custom IValueResolver or inline lambda	Private partial method with matching types — called automatically
Register in DI	AddAutoMapper(typeof(Profile))	AddSingleton<YourMapper>()
Use it	Inject IMapper, call .Map<D>(s)	Inject your mapper class, call the partial method directly